

Stats Binary Frame Structures

Binary frame structures for companion radio stats commands. All multi-byte integers use little-endian byte order.

Command Codes

Command	Code	Description
CMD_GET_STATS	56	Get statistics (2-byte command: code + sub-type)

Stats Sub-Types

The CMD_GET_STATS command uses a 2-byte frame structure: - **Byte 0:** CMD_GET_STATS (56) - **Byte 1:** Stats sub-type: - STATS_TYPE_CORE (0) - Get core device statistics - STATS_TYPE_RADIO (1) - Get radio statistics - STATS_TYPE_PACKETS (2) - Get packet statistics

Response Codes

Response	Code	Description
RESP_CODE_STATS	24	Statistics response (2-byte response: code + sub-type)

Stats Response Sub-Types

The RESP_CODE_STATS response uses a 2-byte header structure: - **Byte 0:** RESP_CODE_STATS (24) - **Byte 1:** Stats sub-type (matches command sub-type): - STATS_TYPE_CORE (0) - Core device statistics response - STATS_TYPE_RADIO (1) - Radio statistics response - STATS_TYPE_PACKETS (2) - Packet statistics response

RESP_CODE_STATS + STATS_TYPE_CORE (24, 0)

Total Frame Size: 11 bytes

Offset	Size	Type	Field Name	Description	Range/Notes
0	1	uint8_t	response_code	Always 0x18 (24)	-
1	1	uint8_t	stats_type	Always 0x00 (STATS_TYPE_CORE)	-
2	2	uint16_t	battery_mv	Battery voltage in millivolts	0 - 65,535
4	4	uint32_t	uptime_secs	Device uptime in seconds	0 - 4,294,967,295
8	2	uint16_t	errors	Error flags bitmask	-
10	1	uint8_t	queue_len	Outbound packet queue length	0 - 255

Example Structure (C/C++)

```
struct StatsCore {
    uint8_t response_code; // 0x18
    uint8_t stats_type;    // 0x00 (STATS_TYPE_CORE)
    uint16_t battery_mv;
    uint32_t uptime_secs;
    uint16_t errors;
    uint8_t queue_len;
} __attribute__((packed));
```

RESP_CODE_STATS + STATS_TYPE_RADIO (24, 1)

Total Frame Size: 14 bytes

Offset	Size	Type	Field Name	Description	Range/Notes
0	1	uint8_t	response_code	Always 0x18 (24)	-
1	1	uint8_t	stats_type	Always 0x01 (STATS_TYPE_RADIO)	-
2	2	int16_t	noise_floor	Radio noise floor in dBm	-140 to +10
4	1	int8_t	last_rssi	Last received signal strength in dBm	-128 to +127
5	1	int8_t	last_snr	SNR scaled by 4	Divide by 4.0 for dB
6	4	uint32_t	tx_air_secs	Cumulative transmit airtime in seconds	0 - 4,294,967,295
10	4	uint32_t	rx_air_secs	Cumulative receive airtime in seconds	0 - 4,294,967,295

Example Structure (C/C++)

```
struct StatsRadio {
    uint8_t response_code; // 0x18
    uint8_t stats_type;    // 0x01 (STATS_TYPE_RADIO)
    int16_t noise_floor;
    int8_t last_rssi;
    int8_t last_snr;       // Divide by 4.0 to get actual SNR in dB
    uint32_t tx_air_secs;
    uint32_t rx_air_secs;
} __attribute__((packed));
```

RESP_CODE_STATS + STATS_TYPE_PACKETS (24, 2)

Total Frame Size: 26 bytes (legacy) or 30 bytes (includes recv_errors)

Offset	Size	Type	Field Name	Description	Range/Notes
0	1	uint8_t	response_code	Always 0x18 (24)	-
1	1	uint8_t	stats_type	Always 0x02 (STATS_TYPE_PACKETS)	-
2	4	uint32_t	recv	Total packets received	0 - 4,294,967,295
6	4	uint32_t	sent	Total packets sent	0 - 4,294,967,295
10	4	uint32_t	flood_tx	Packets sent via flood routing	0 - 4,294,967,295
14	4	uint32_t	direct_tx	Packets sent via direct routing	0 - 4,294,967,295
18	4	uint32_t	flood_rx	Packets received via flood routing	0 - 4,294,967,295
22	4	uint32_t	direct_rx	Packets received via direct routing	0 - 4,294,967,295
26	4	uint32_t	recv_errors	Receive/CRC errors (RadioLib); present only in 30-byte frame	0 - 4,294,967,295

Notes

- Counters are cumulative from boot and may wrap.
- `recv` = `flood_rx` + `direct_rx`
- `sent` = `flood_tx` + `direct_tx`
- Clients should accept frame length ≥ 26 ; if length ≥ 30 , parse `recv_errors` at offset 26.

Example Structure (C/C++)

```
struct StatsPackets {
    uint8_t response_code; // 0x18
    uint8_t stats_type;    // 0x02 (STATS_TYPE_PACKETS)
```

```

uint32_t recv;
uint32_t sent;
uint32_t flood_tx;
uint32_t direct_tx;
uint32_t flood_rx;
uint32_t direct_rx;
uint32_t recv_errors;    // present when frame size is 30
} __attribute__((packed));

```

Command Usage Example (Python)

```

# Send CMD_GET_STATS command
def send_get_stats_core(serial_interface):
    """Send command to get core stats"""
    cmd = bytes([56, 0]) # CMD_GET_STATS (56) + STATS_TYPE_CORE (0)
    serial_interface.write(cmd)

def send_get_stats_radio(serial_interface):
    """Send command to get radio stats"""
    cmd = bytes([56, 1]) # CMD_GET_STATS (56) + STATS_TYPE_RADIO (1)
    serial_interface.write(cmd)

def send_get_stats_packets(serial_interface):
    """Send command to get packet stats"""
    cmd = bytes([56, 2]) # CMD_GET_STATS (56) + STATS_TYPE_PACKETS (2)
    serial_interface.write(cmd)

```

Response Parsing Example (Python)

```

import struct

def parse_stats_core(frame):
    """Parse RESP_CODE_STATS + STATS_TYPE_CORE frame (11 bytes)"""
    response_code, stats_type, battery_mv, uptime_secs, errors, queue_len = \
        struct.unpack('<B B H I H B', frame)
    assert response_code == 24 and stats_type == 0, "Invalid response type"
    return {
        'battery_mv': battery_mv,
        'uptime_secs': uptime_secs,
        'errors': errors,
        'queue_len': queue_len
    }

def parse_stats_radio(frame):
    """Parse RESP_CODE_STATS + STATS_TYPE_RADIO frame (14 bytes)"""
    response_code, stats_type, noise_floor, last_rssi, last_snr, tx_air_secs, \
    rx_air_secs = \
        struct.unpack('<B B h b b I I', frame)
    assert response_code == 24 and stats_type == 1, "Invalid response type"
    return {
        'noise_floor': noise_floor,
        'last_rssi': last_rssi,
        'last_snr': last_snr / 4.0, # Unscale SNR
        'tx_air_secs': tx_air_secs,
        'rx_air_secs': rx_air_secs
    }

```

```
def parse_stats_packets(frame):
    """Parse RESP_CODE_STATS + STATS_TYPE_PACKETS frame (26 or 30 bytes)"""
    assert len(frame) >= 26, "STATS_TYPE_PACKETS frame too short"
    response_code, stats_type, recv, sent, flood_tx, direct_tx, flood_rx,
    direct_rx = \
        struct.unpack('<B B I I I I I I', frame[:26])
    assert response_code == 24 and stats_type == 2, "Invalid response type"
    result = {
        'recv': recv,
        'sent': sent,
        'flood_tx': flood_tx,
        'direct_tx': direct_tx,
        'flood_rx': flood_rx,
        'direct_rx': direct_rx
    }
    if len(frame) >= 30:
        (recv_errors,) = struct.unpack('<I', frame[26:30])
        result['recv_errors'] = recv_errors
    return result
```

Command Usage Example (JavaScript/TypeScript)

```
// Send CMD_GET_STATS command
const CMD_GET_STATS = 56;
const STATS_TYPE_CORE = 0;
const STATS_TYPE_RADIO = 1;
const STATS_TYPE_PACKETS = 2;

function sendGetStatsCore(serialInterface: SerialPort): void {
    const cmd = new Uint8Array([CMD_GET_STATS, STATS_TYPE_CORE]);
    serialInterface.write(cmd);
}

function sendGetStatsRadio(serialInterface: SerialPort): void {
    const cmd = new Uint8Array([CMD_GET_STATS, STATS_TYPE_RADIO]);
    serialInterface.write(cmd);
}

function sendGetStatsPackets(serialInterface: SerialPort): void {
    const cmd = new Uint8Array([CMD_GET_STATS, STATS_TYPE_PACKETS]);
    serialInterface.write(cmd);
}
```

Response Parsing Example (JavaScript/TypeScript)

```
interface StatsCore {
    battery_mv: number;
    uptime_secs: number;
    errors: number;
    queue_len: number;
}

interface StatsRadio {
    noise_floor: number;
    last_rssi: number;
    last_snr: number;
    tx_air_secs: number;
}
```

```

    rx_air_secs: number;
}

interface StatsPackets {
    recv: number;
    sent: number;
    flood_tx: number;
    direct_tx: number;
    flood_rx: number;
    direct_rx: number;
    recv_errors?: number; // present when frame is 30 bytes
}

function parseStatsCore(buffer: ArrayBuffer): StatsCore {
    const view = new DataView(buffer);
    const response_code = view.getUint8(0);
    const stats_type = view.getUint8(1);
    if (response_code !== 24 || stats_type !== 0) {
        throw new Error('Invalid response type');
    }
    return {
        battery_mv: view.getUint16(2, true),
        uptime_secs: view.getUint32(4, true),
        errors: view.getUint16(8, true),
        queue_len: view.getUint8(10)
    };
}

function parseStatsRadio(buffer: ArrayBuffer): StatsRadio {
    const view = new DataView(buffer);
    const response_code = view.getUint8(0);
    const stats_type = view.getUint8(1);
    if (response_code !== 24 || stats_type !== 1) {
        throw new Error('Invalid response type');
    }
    return {
        noise_floor: view.getInt16(2, true),
        last_rssi: view.getInt8(4),
        last_snr: view.getInt8(5) / 4.0, // Unscale SNR
        tx_air_secs: view.getUint32(6, true),
        rx_air_secs: view.getUint32(10, true)
    };
}

function parseStatsPackets(buffer: ArrayBuffer): StatsPackets {
    const view = new DataView(buffer);
    if (buffer.byteLength < 26) {
        throw new Error('STATS_TYPE_PACKETS frame too short');
    }
    const response_code = view.getUint8(0);
    const stats_type = view.getUint8(1);
    if (response_code !== 24 || stats_type !== 2) {
        throw new Error('Invalid response type');
    }
    const result: StatsPackets = {
        recv: view.getUint32(2, true),
        sent: view.getUint32(6, true),
        flood_tx: view.getUint32(10, true),
        direct_tx: view.getUint32(14, true),
        flood_rx: view.getUint32(18, true),
        direct_rx: view.getUint32(22, true)
    };
}

```

```
};  
if (buffer.byteLength >= 30) {  
    result.recv_errors = view.getUint32(26, true);  
}  
return result;  
}
```

Field Size Considerations

- Packet counters (uint32_t): May wrap after extended high-traffic operation.
- Time fields (uint32_t): Max ~136 years.
- SNR (int8_t, scaled by 4): Range -32 to +31.75 dB, 0.25 dB precision.